Screenshots are below the deployment instructions

Deployment

This guide is for someone who lands on the repository with no prior context and needs to go from a downloaded ZIP file to a working full-stack application.

Repository

GitHub repository: https://github.com/NicolasRossetti/CSCE548-Spr2026
The project has two runnable parts:

- Backend: Spring Boot service with service, business, and data layers
- Frontend: React plus Vite client in the `client` folder

What You Are Deploying

Backend layers:

- Data layer: src/main/java/com/bazaar/dao/BazaarDAO.java
- Business layer: src/main/java/com/bazaar/business/BazaarBusinessService.java and src/main/java/com/bazaar/business/BazaarBusinessServiceImpl.java
- Service layer: src/main/java/com/bazaar/api/*

Frontend layer:

- Web client: client/src/App.jsx

Database scripts:

- Schema: sql/schema.sql
- Starter data: sql/test_data.sql
- Verification queries: sql/project4_verification_queries.sql

Verification helpers:

- Console CRUD client: src/main/java/com/bazaar/ServiceConsoleClient.java
- End-to-end PowerShell script: scripts/project4_system_test.ps1

Prerequisites

Required Software

Install these before you start.

1. Java Development Kit 17
2. Maven 3.9 or newer
3. Node.js 18 or newer
4. npm 9 or newer
5. MySQL Server 8.x
6. A MySQL client tool
7. PowerShell 7 or newer if you want to run the provided system test script

Recommended Tools

1. Visual Studio Code or IntelliJ IDEA for editing and running the project
2. Git if you prefer cloning instead of downloading the ZIP
3. Railway account if you want to host the backend online
4. A static hosting platform such as Netlify or Vercel if you want to host the frontend online

Default Ports and Runtime Settings

- Backend HTTP port: `8080`
- Frontend dev server port: `5173`
- Default local database name: `bazaar_tracker`
- Default local frontend to backend target: `http://localhost:8080`

Part 1: Download and Unpack the Repository

Option A: Download ZIP from GitHub

1. Open the repository page in your browser.
2. Click the green “Code” button.
3. Click “Download ZIP”.
4. Save the ZIP file to a folder you control, such as your Desktop or Documents folder.
5. Extract the ZIP file. On Windows you can right-click the ZIP and choose “Extract All”.
6. Rename the extracted folder if you want, then open that folder in your IDE.

Option B: Clone with Git

In powershell, run the following commands:

```
git clone https://github.com/NicolasRossetti/CSCE548-Spr2026.git
cd CSCE548-Spr2026
```

Part 2: Open the Project in an IDE

Open the extracted project root, not only the `client` folder.
You should see these top-level directories:

- src
- client
- sql
- scripts
- docs

If you are using VS Code, open a terminal at the repository root before running backend commands.

Part 3: Set Up the Database

The backend will not work until MySQL is running and the schema exists.

Create the Database Schema

Start MySQL Server, then run this command in powershell:

```
mysql -u root -p < sql/schema.sql
```

That script creates:

- bazaar_tracker
- items
- price_snapshots
- orders
- trades
- Notes

Load Starter Data

Load the sample data so the app has records to display immediately:

In powershell:

```
mysql -u root -p < sql/test_data.sql
```

Verify the Database Setup

Open MySQL and run:

```
USE bazaar_tracker;
SHOW TABLES;
SELECT COUNT(*) AS item_count FROM items;
```

Expected result:

- You should see all 5 tables.
- “items” should contain starter rows after “test_data.sql” is loaded.

Part 4: Configure the Backend

The backend reads database settings from environment variables.

Required Backend Environment Variables

```
$env:BAZAAR_DB_URL="jdbc:mysql://localhost:3306/bazaar_tracker"
$env:BAZAAR_DB_USERNAME="root"
$env:BAZAAR_DB_PASSWORD="your_mysql_password"
```

If your MySQL installation requires extra JDBC options, use this form instead:

```
$env:BAZAAR_DB_URL="jdbc:mysql://localhost:3306/bazaar_tracker?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC"
```

Optional Backend Environment Variables

For local frontend access, the default CORS setting is already correct. If you host the frontend somewhere else, set this too:

```
$env:BAZAAR_CORS_ALLOWED_ORIGINS="http://localhost:5173,https://your-frontend-domain"
```

Part 5: Build the Backend

From the repository root, run:

```
mvn clean package
```

What this does:

- Compiles the Java source
- Resolves Maven dependencies
- Builds the Spring Boot executable JAR

Expected output:

- Build ends with “BUILD SUCCESS”
- Output JAR appears at “target/bazaar-tracker-1.0.0.jar”

Part 6: Run the Backend Locally

You can run the backend in either of these ways.

Method A: Run from Maven

```
mvn spring-boot:run
```

Method B: Run the Packaged JAR

```
java -jar target/bazaar-tracker-1.0.0.jar
```

Expected result:
- Spring Boot starts successfully
- The service listens on `http://localhost:8080`

How to Know the Backend Works

Open one of these URLs in your browser or call them from PowerShell:

- http://localhost:8080/api/items
- http://localhost:8080/api/orders
- http://localhost:8080/api/notes

Example PowerShell check:

```
Invoke-RestMethod http://localhost:8080/api/items
```

Expected result:

- You receive JSON back.
- If you loaded “sql/test_data.sql”, “GET /api/items” should return multiple records.

If the app starts but requests fail with a database error, double-check the three “BAZAAR_DB_*” environment variables.

Part 7: Set Up and Run the Frontend Locally

Open a second terminal.

Install Client Dependencies

In powershell:

```
cd client
npm install
```

Start the Frontend Dev Server

In powershell:

```
npm run dev
```

Expected result:

- Vite starts on `http://localhost:5173`

How to Know the Frontend Works

1. Open “http://localhost:5173” in your browser.
2. Confirm the page shows the Bazaar Tracker Client UI.
3. Click “Run All Required GET Calls”.
4. Confirm that each entity section logs successful GET activity.

If the backend contains starter data, the UI should display rows for:

- Items
- Price Snapshots
- Orders
- Trades
- Notes

The connection panel accepts either:

- A local port such as `8080`
- A full backend URL such as `https://your-backend.up.railway.app`

Part 8: Verify the Entire Stack End to End

Use at least one of these verification methods.

Option A: Browser Verification

1. Backend returns JSON from `http://localhost:8080/api/items`
2. Frontend loads at `http://localhost:5173`
3. Frontend can load records for all five entity types

Option B: Java Console CRUD Client

From the repository root while the backend is running (in powershell):

```
mvn exec:java -Dexec.mainClass=com.bazaar.ServiceConsoleClient
```

Expected result:

- The console prints create, read, update, and delete activity for all five entity types.
- The final delete-verification requests return `404`, which confirms the rows were removed.

Option C: PowerShell Full System Test

From the repository root while the backend is running:

```
pwsh ./scripts/project4_system_test.ps1
```

Expected result:

- The script prints `Project 4 system test completed successfully.`
- A JSON artifact is written to the `artifacts` folder.

If you want to inspect created rows before deletion, run:

```
pwsh ./scripts/project4_system_test.ps1 -SkipDelete
```

Then use `sql/project4_verification_queries.sql` to inspect the created records directly in MySQL.

Part 9: Host the Backend Online on Railway

This is the recommended cloud-hosting workflow for the Spring Boot service.

Prepare the Repository

1. Push the project to GitHub if you are working from a local copy.
2. Make sure the root contains `pom.xml`.

Create the Railway Project

1. Sign in to Railway.
2. Create a new project from your GitHub repository.
3. Add a MySQL service to the Railway project.

Configure Railway Environment Variables for the Java Service

Set these in the Java service:

```
BAZAAR_DB_URL=jdbc:mysql://mysql.railway.internal:3306/railway?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
BAZAAR_DB_USERNAME=root
BAZAAR_DB_PASSWORD=<value from Railway MySQL service>
```

If you plan to host the frontend on a different domain, also set:

```
BAZAAR_CORS_ALLOWED_ORIGINS=https://your-frontend-domain,http://localhost:5173
```

Railway Build and Start Commands

- Build command: `mvn clean package`
- Start command: `java -jar target/bazaar-tracker-1.0.0.jar`

Initialize the Railway Database

Connect to the Railway MySQL service from a MySQL client, then run:

```
SOURCE sql/schema.sql;
SOURCE sql/test_data.sql;
```

If your SQL tool does not support `SOURCE`, open each file and run its contents manually.

How to Know the Railway Backend Works

1. Railway deployment logs show the Spring Boot app started.
2. Visiting `https://your-railway-service.up.railway.app/api/items` returns JSON.
3. Running the system test against Railway succeeds:

In powershell:

```
$env:BAZAAR_API_URL="https://your-railway-service.up.railway.app/api”
pwsh ./scripts/project4_system_test.ps1
```

Part 10: Host the Frontend Online

The frontend is a static Vite build. It can be hosted on any static host.

Recommended static-host settings:

- Project root for the frontend host: `client`
- Install command: `npm install`
- Build command: `npm run build`
- Publish directory: `dist`
- Environment variable: `VITE_API_BASE_URL=https://your-backend.up.railway.app`

If the frontend is deployed on a different domain from the backend, the backend must allow that domain through CORS.

Example:

```
BAZAAR_CORS_ALLOWED_ORIGINS=https://your-frontend-domain,http://localhost:5173
```

How to Know the Hosted Frontend Works

1. The hosted page loads without a blank screen.
2. The client can load `Items`, `Orders`, and the other entity tabs.
3. “Run All Required GET Calls” completes without network errors.
4. Browser developer tools show successful requests to your hosted backend URL.

Common Troubleshooting

Backend starts but API returns database errors

Cause:

- Wrong `BAZAAR_DB_URL`, `BAZAAR_DB_USERNAME`, or `BAZAAR_DB_PASSWORD`

Fix:

- Recheck the environment variables in the same terminal or hosting dashboard where the backend is running.

API returns an empty list

Cause:

- You created the schema but did not load `sql/test_data.sql`

Fix:

- Run the starter-data script, then call the API again.

Frontend cannot reach the backend

Cause:

- Wrong local port or wrong hosted URL in the connection settings or `VITE_API_BASE_URL`

Fix:

- In the client connection panel, enter `8080` for local use or the full deployed backend URL for hosted use.
- Do not include `/api` in the saved value.

Browser shows a CORS error

Cause:

- The frontend origin is not included in `BAZAAR_CORS_ALLOWED_ORIGINS`

Fix:

- Add the frontend domain and redeploy the backend.

`pwsh ./scripts/project4_system_test.ps1` fails immediately

Cause:

- PowerShell 7 is not installed, or the backend URL is wrong

Fix:

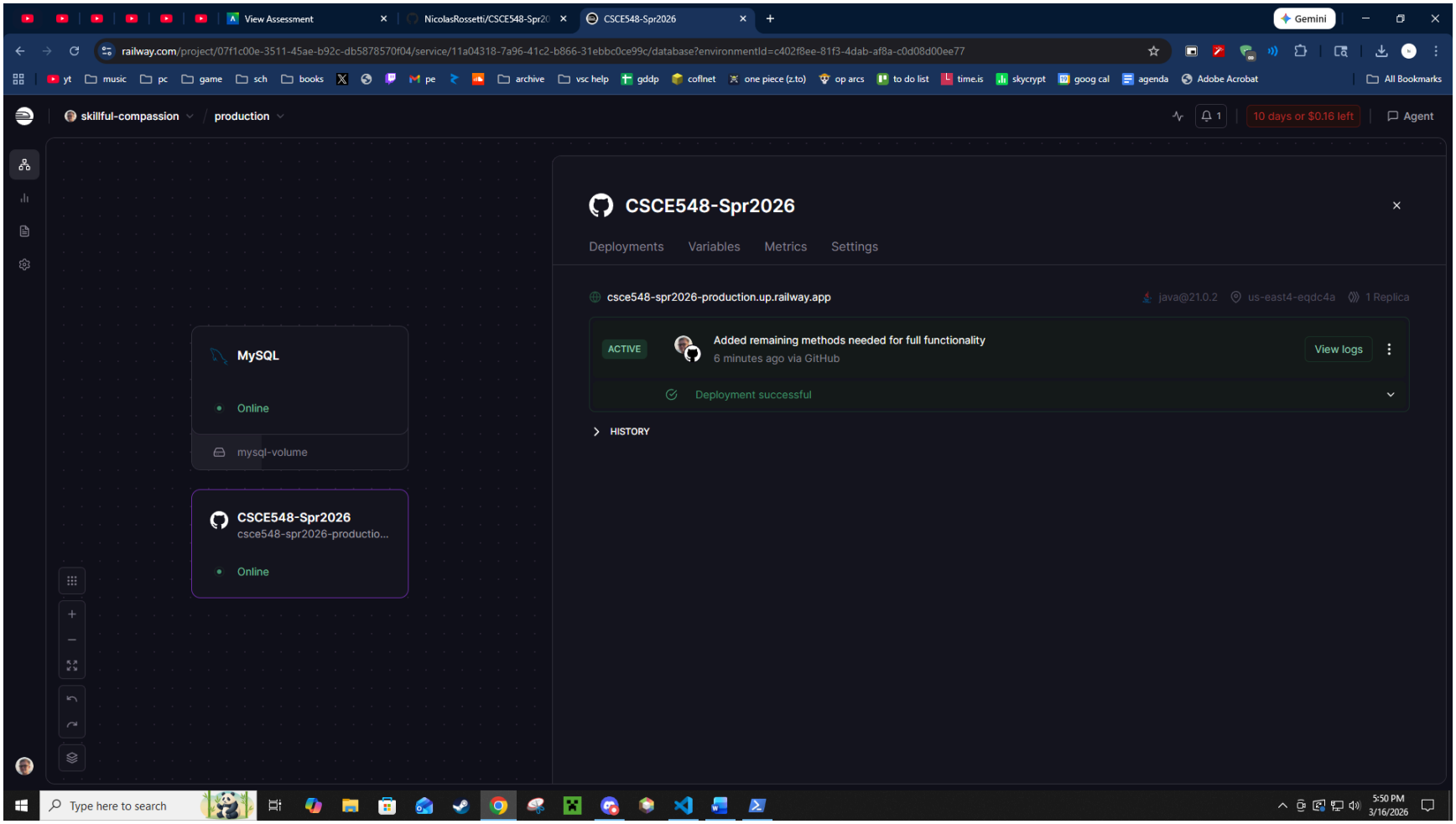
- Run the script in `pwsh`, not Windows PowerShell 5.x.
- Confirm `BAZAAR_API_URL` points at the correct backend.

Quick Success Checklist

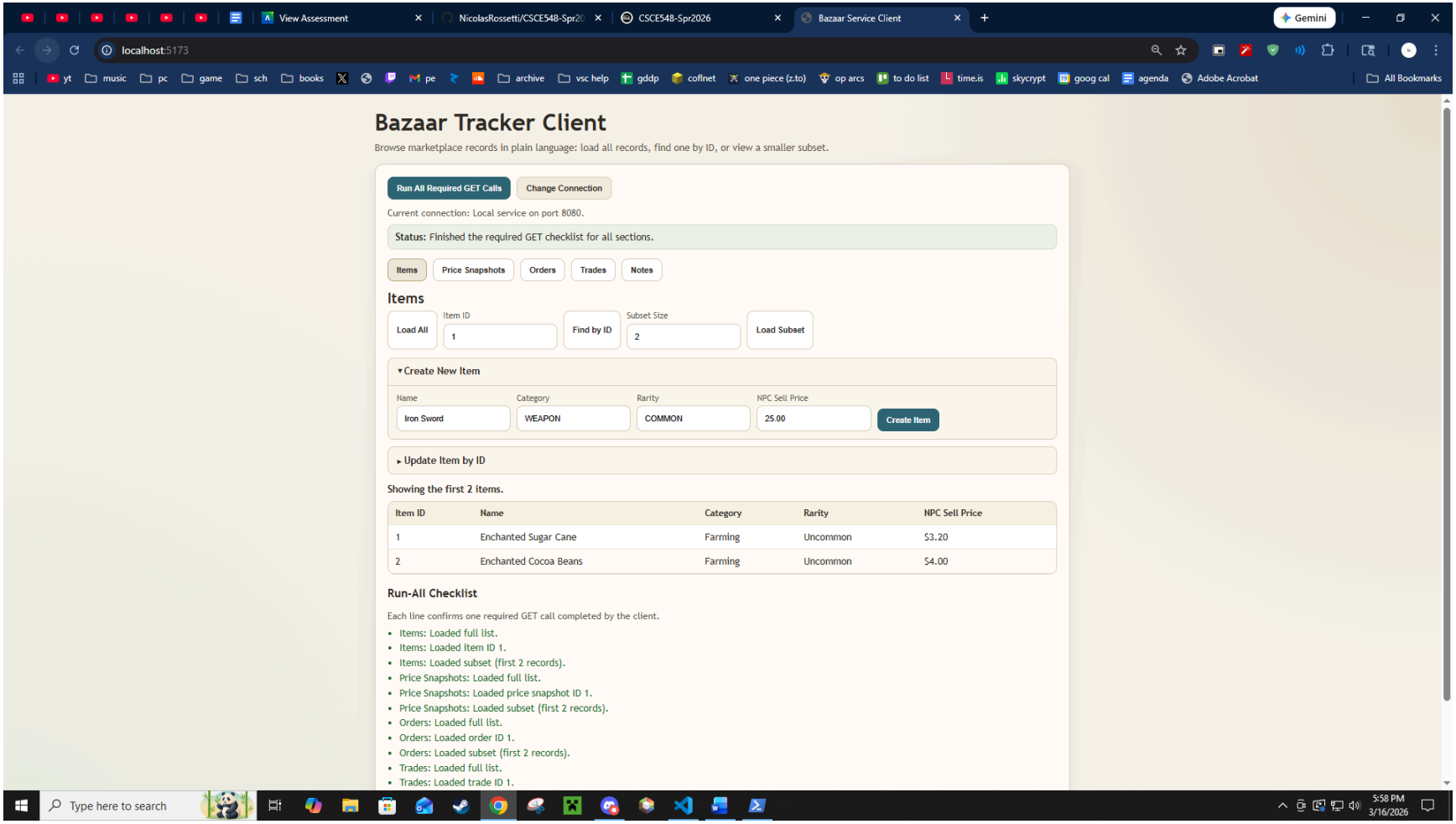
You are done when all of these are true:

1. MySQL contains the `bazaar_tracker` schema and all five tables.
2. `mvn clean package` succeeds.
3. `http://localhost:8080/api/items` or your hosted backend URL returns JSON.
4. `npm run dev` or your hosted frontend serves the client UI.
5. The frontend can load data for all five entity types.
6. The console client or PowerShell system test completes successfully.

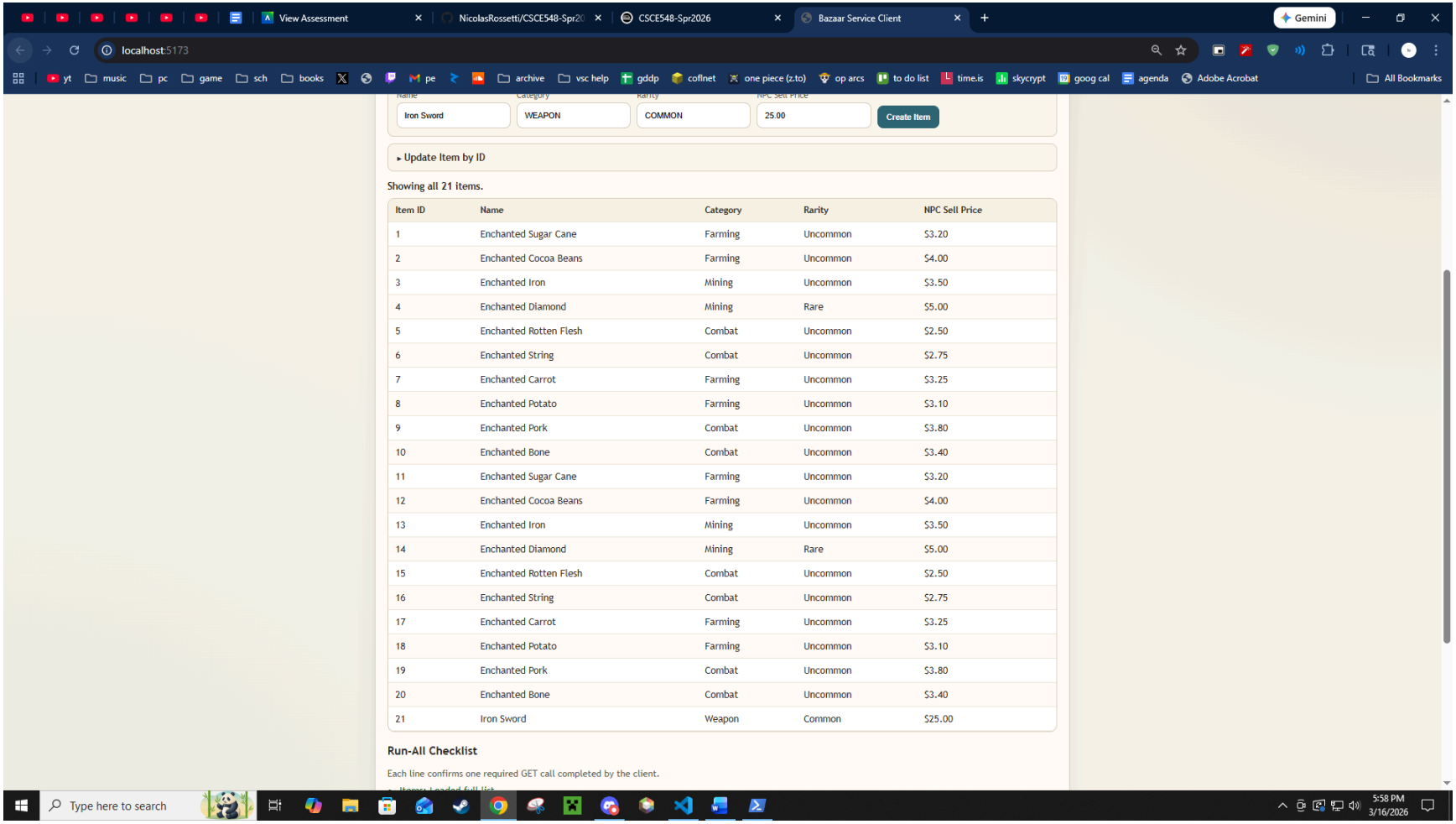
Proof of platform hosting database



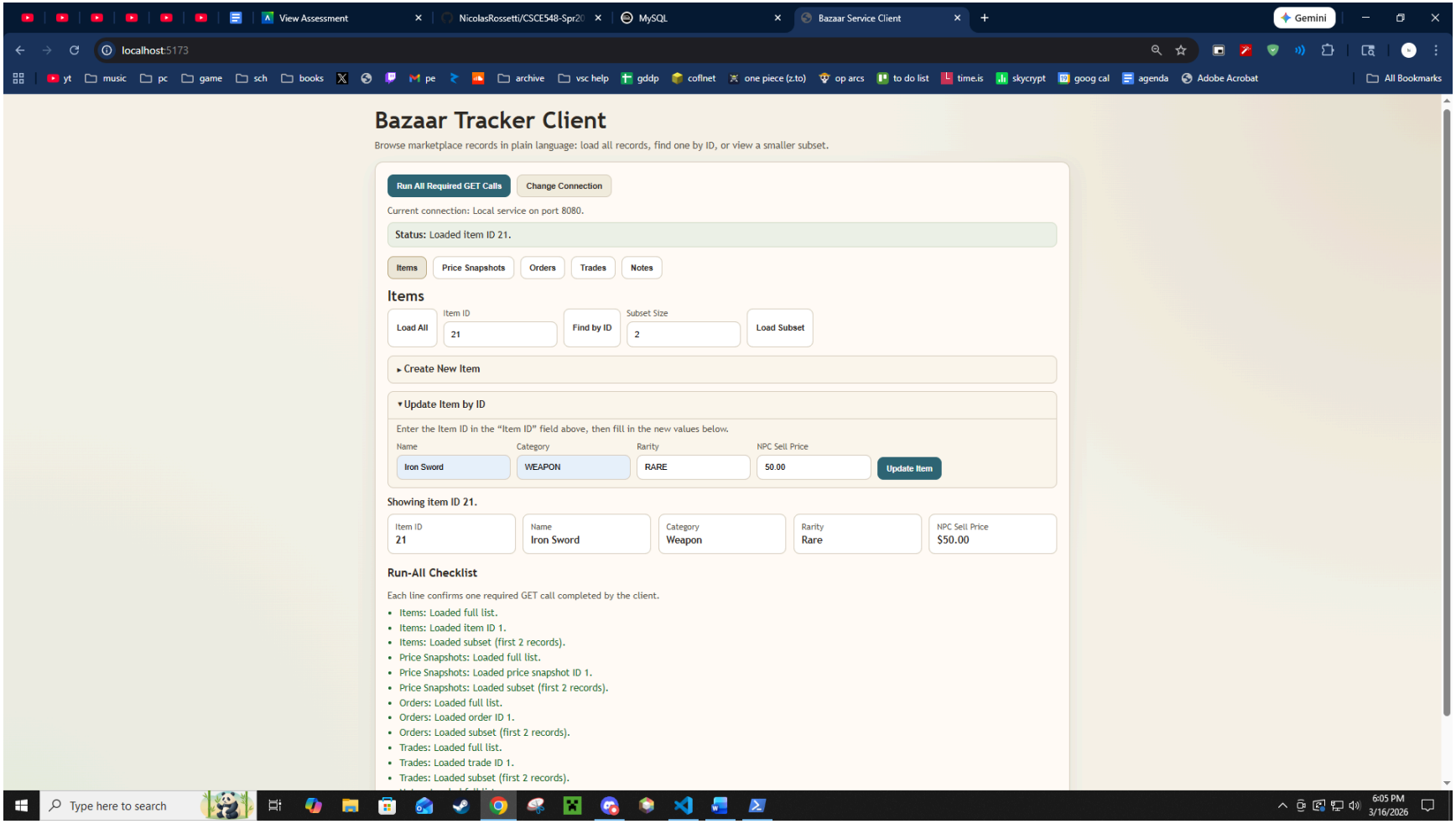
Adding an entry to “Items” table (ID 21)



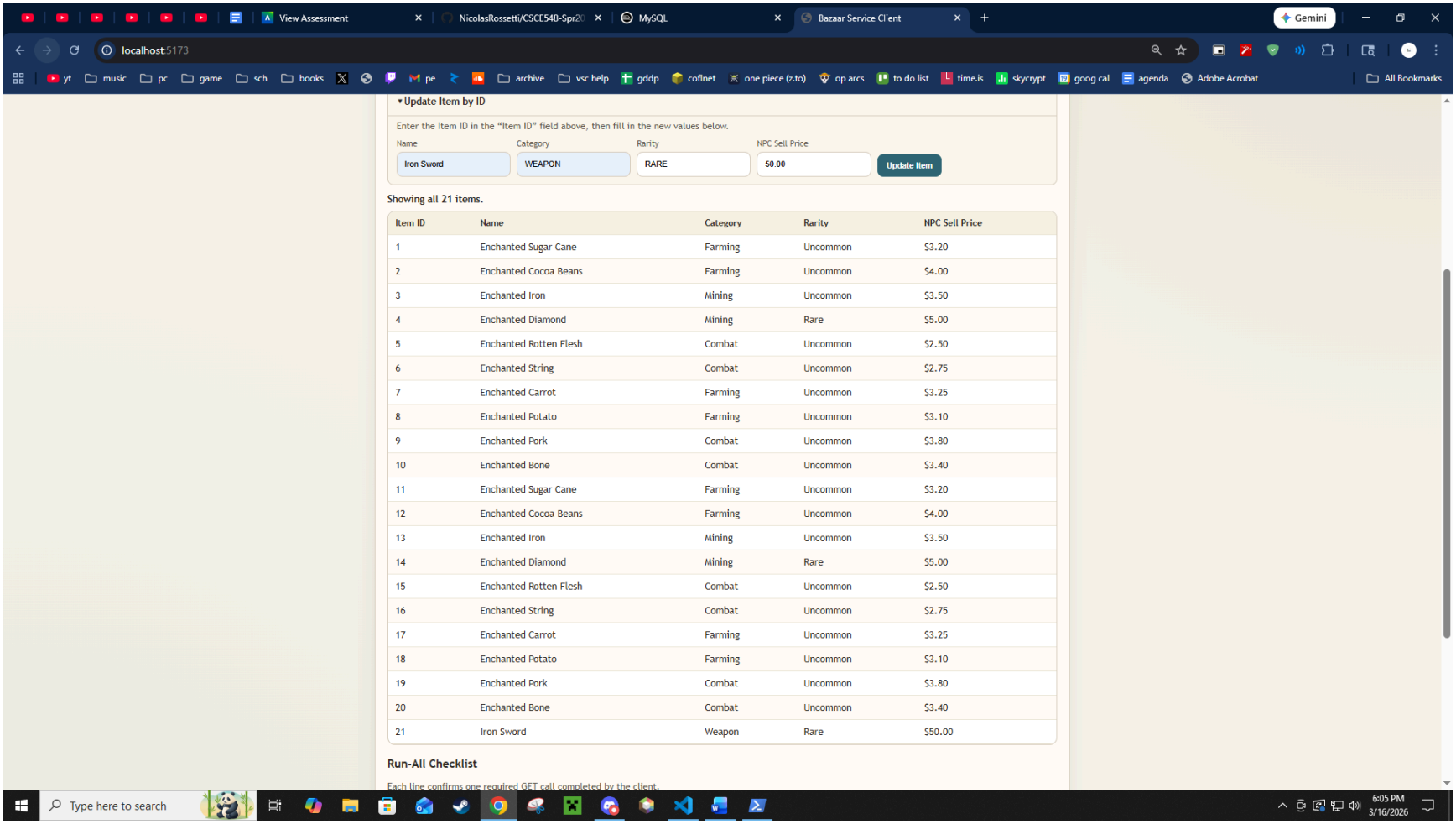
Showing added entry to “Items” table (ID 21)



Updating an entry in “Items” table (ID 21)

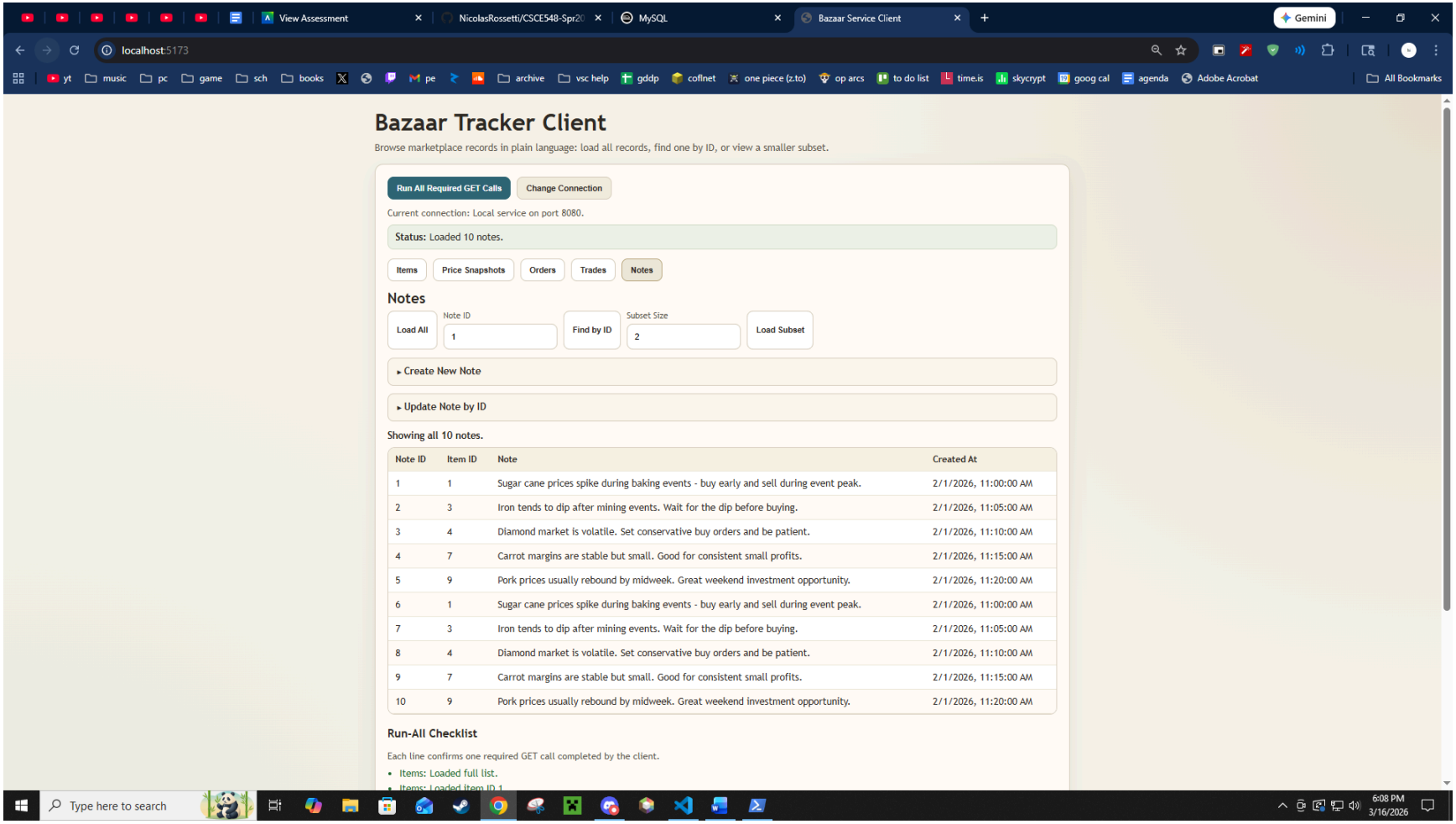


Showing updated entry in “Items” table (ID 21)

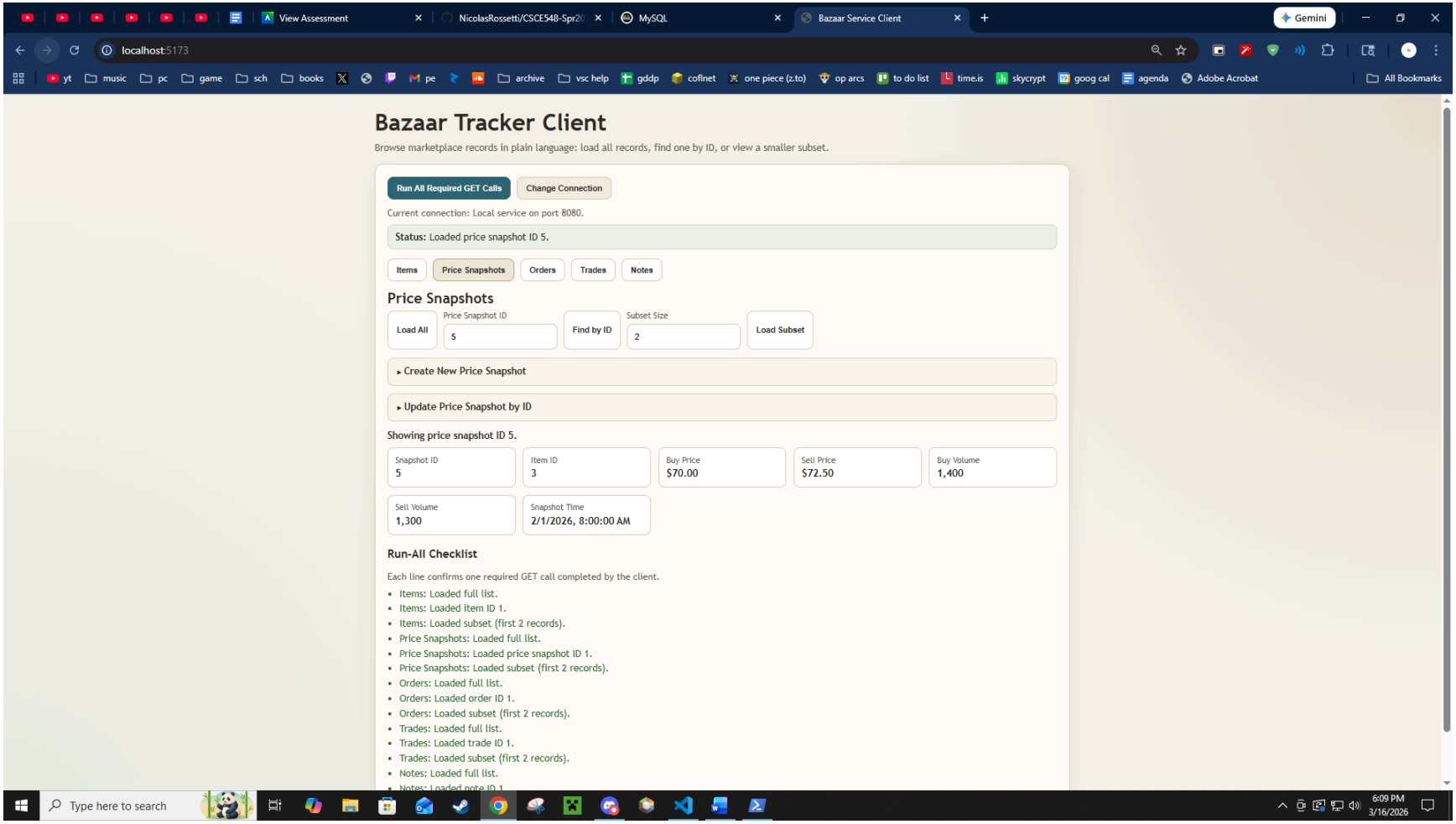
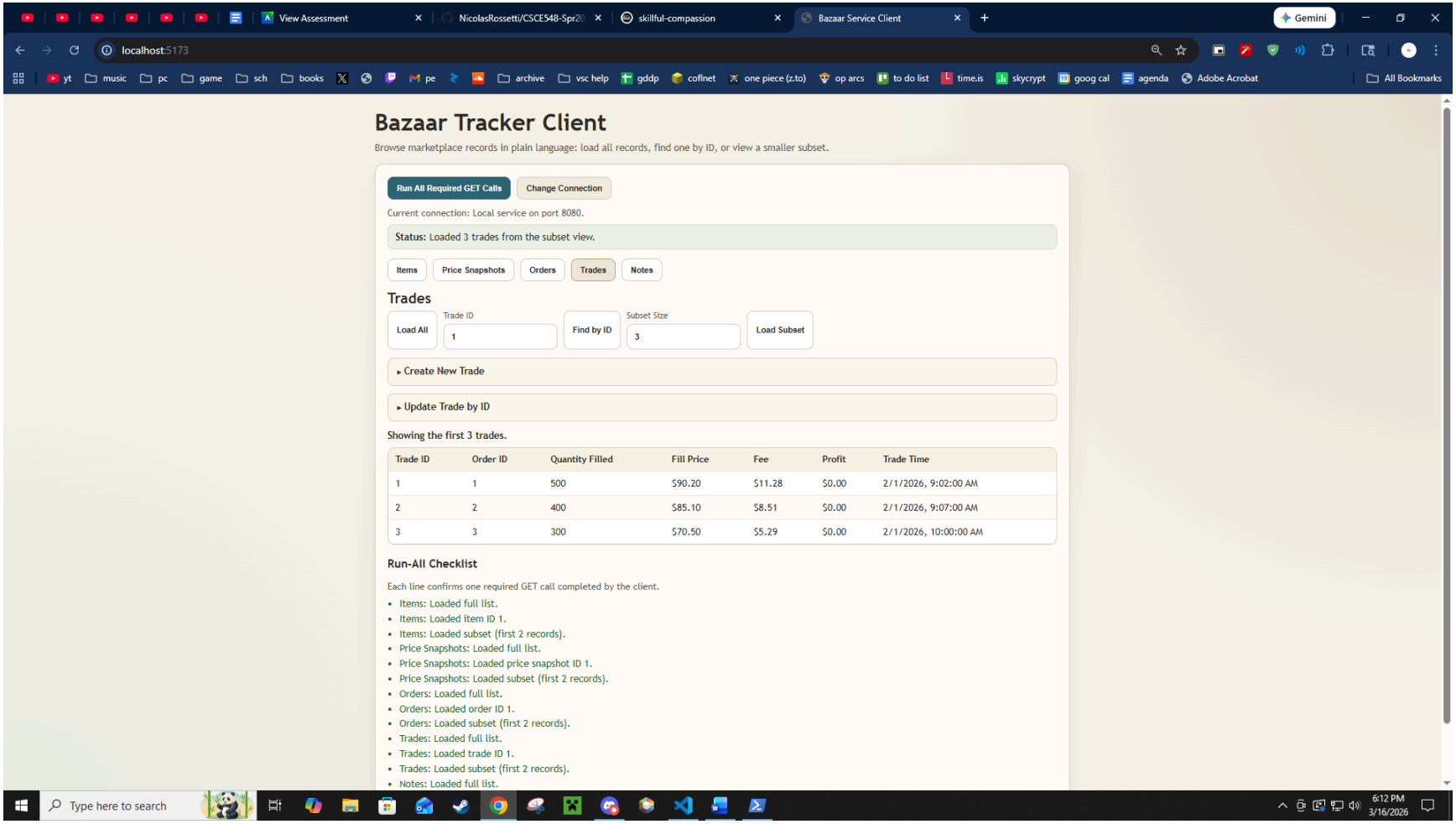


Item ID	Name	Category	Rarity	NPC Sell Price
1	Enchanted Sugar Cane	Farming	Uncommon	$3.20
2	Enchanted Cocoa Beans	Farming	Uncommon	$4.00
3	Enchanted Iron	Mining	Uncommon	$3.50
4	Enchanted Diamond	Mining	Rare	$5.00
5	Enchanted Rotten Flesh	Combat	Uncommon	$2.50
6	Enchanted String	Combat	Uncommon	$2.75
7	Enchanted Carrot	Farming	Uncommon	$3.25
8	Enchanted Potato	Farming	Uncommon	$3.10
9	Enchanted Pork	Combat	Uncommon	$3.80
10	Enchanted Bone	Combat	Uncommon	$3.40
11	Enchanted Sugar Cane	Farming	Uncommon	$3.20
12	Enchanted Cocoa Beans	Farming	Uncommon	$4.00
13	Enchanted Iron	Mining	Uncommon	$3.50
14	Enchanted Diamond	Mining	Rare	$5.00
15	Enchanted Rotten Flesh	Combat	Uncommon	$2.50
16	Enchanted String	Combat	Uncommon	$2.75
17	Enchanted Carrot	Farming	Uncommon	$3.25
18	Enchanted Potato	Farming	Uncommon	$3.10
19	Enchanted Pork	Combat	Uncommon	$3.80
20	Enchanted Bone	Combat	Uncommon	$3.40
21	Iron Sword	Weapon	Rare	$50.00

Getting all entries in “Notes” table (getting all items)



Getting a single entry (ID 5) from “Price Snapshots” table



Getting a subset of entries from “Trades” table